

Roof PathFinder Project Specification

Project – Roof

Data Foundation Lab
DFLab FHO Pty Ltd – Proprietary Limited Company
(13 676 090 289)

Table of Contents

1. Overview *4*

1.1 Purpose 4

1.2 Scope 4

2. Functional Responsibilities *5*

2.1 Out-of-Scope Responsibilities 6

3. System Architecture *6*

3.1 High-Level Architecture 6

3.2 Process Startup 6

3.3 Relay Runtime Model 7

3.4 Client Handling Flow 7

3.5 Client Registry Model 8

3.5.1 Temporary Connection Registry 8

3.5.2 Identified Client Registry 8

3.6 Main Modules 9

4. Data Model *9*

4.1 PathFinderClient 9

4.2 RoofWebSocketPacket 10

4.3 Native Request Packet 10

4.4 Native Response Packet 10

5. Communication Model *11*

5.1 General Communication Flow 11

5.2 Connection Binding 11

5.3 Duplicate Identifier Behaviour 11

5.4 Routing Flow 12

5.5 Payload Handling 12

6. PathFinder Networking Interface *12*

6.1 Listener Configuration 12

6.2 WebSocket Handshake Requirements 13

6.3 Cloudflare Header Handling 13

6.4 Packet Validation 13

6.5 Unsupported Message Types 14

7. Heartbeat & Disconnection Model *14*

7.1 Constants 14

7.2 Relay-to-Client Heartbeat 14

7.3 Client-to-Relay Heartbeat 15
7.4 Disconnection Detection 15
8. Security & Trust Boundaries 16
8.1 Relay Security Position 16
8.2 Payload Confidentiality 16
8.3 SenderFingerprint 17
8.4 TLS and Proxy Boundary 17
8.5 Security Direction Consideration 18
9. Networking Protocols 18
10. Scalability Model 18
10.1 Current Scalability Model 18
10.2 Horizontal Scaling Considerations 19
10.3 Load Distribution 19
10.4 Connection Limits 19
11. Failure Handling 20
11.1 Startup Failure 20
11.2 Handshake Failure 20
11.3 Packet Receive Failure 20
11.4 Routing Failure 20
11.5 Client Close Path 21
11.6 Heartbeat Timeout Path 21
11.7 Shutdown Behaviour 21
12. Performance Considerations 22
12.1 Latency 22
12.2 Throughput 22
12.3 Heartbeat Overhead 23
12.4 Heartbeat Loop Scheduling 23
12.5 Packet Size 23
13. Deployment & Runtime Model 24
13.1 Pathfinder Project Configuration 24
13.2 Network Exposure 24
13.3 Update Strategy 25
13.4 Versioning 25
14. Observability 26
14.1 Logging Utility 26

14.2 PathFinder Log Events **26**
14.3 Metrics Consideration **27**
14.4 Payload Logging Boundary **27**
15. Testing & Validation ***27***
15.1 Validation Areas **27**
15.2 Network Simulation **28**
15.3 Load Testing **28**
15.4 Acceptance Criteria **29**
16. Risks & Limitations ***29***
16.1 Security Risks **29**
16.2 Reliability Risks **30**
16.3 Performance Risks **30**
16.4 Protocol Compatibility Risks **30**
16.5 Implementation Completeness Risks **30**
Appendices ***32***
Appendix A: Important Constants **32**
Appendix B: Glossary **33**
Appendix C: Relay Connection Flow **34**
Appendix D: Heartbeat Flow **34**
Appendix E: Component Diagram **34**

Version: V1.0.0 (Major, Minor, Patch)
Status: ON-GOING
Owner: Steve Yoo
Last Updated: 09/11/2026

---

1. Overview

Roof PathFinder is the remote relay component used by Roof components when direct local-network communication is unavailable or unsuitable. Its role is to provide a WebSocket-based relay pathway between Roof clients, devices, and services that need to communicate across network boundaries.

PathFinder is not intended to own user data, inspect application payloads, or act as a central authority over Roof data. Its function is limited to connection coordination, client presence tracking, heartbeat handling, and forwarding relay packets between registered endpoints.

PathFinder currently operates as a .NET 10 NativeAOT executable. It listens for raw TCP connections on port 8071, performs a manual WebSocket upgrade handshake, tracks clients by Roof identifier, and forwards RoofWebSocketPacket messages between connected clients.

Roof Common is the shared support library used by PathFinder and other Roof components. In the PathFinder context, Roof Common provides the packet contract, JSON serialisation contexts, RNMS provider abstraction, heartbeat behaviour, logging helpers, and security/hash utilities.

1.1 Purpose

The purpose of PathFinder is to provide a remote relay layer for Roof communication when two Roof components cannot communicate directly over the local network.

PathFinder supports the wider Roof infrastructure by:

- enabling WebSocket relay communication;
- maintaining a registry of connected Roof clients;
- forwarding packets between active registered endpoints;
- providing heartbeat-based liveness detection;
- preserving a relay-only model where PathFinder does not inspect or process application payload content.

1.2 Scope

This specification covers:

- Roof PathFinder process responsibilities;
- relay startup and shutdown lifecycle;
- WebSocket handshake behaviour;
- connection registration and routing;
- packet model and serialisation;
- heartbeat and timeout behaviour;

- Roof Common RNMS provider responsibilities relevant to PathFinder;
- security boundaries;
- logging and observability;
- deployment assumptions;
- known implementation risks and future decisions.

This specification does NOT fully define:

- Roof Engine application-level behaviour;
- RoofDeck user interface behaviour;
- end-to-end encryption policy beyond PathFinder's relay boundary;
- production infrastructure topology outside PathFinder;
- final authentication and authorisation model for relay clients.

2. Functional Responsibilities

PathFinder is responsible for operating the remote WebSocket relay process.

Its current functional responsibilities are:

- run as a .NET 10 executable with NativeAOT enabled;
- start a background WebSocketRelay thread from Program.Main;
- keep the process alive until Ctrl+C or process exit;
- listen for incoming WebSocket-over-TCP connections on IPAddress.Any:8071;
- manually perform the WebSocket HTTP upgrade handshake;
- extract optional Cloudflare headers, including:
    - CF-Connecting-IP;
    - CF-Ray;
    - Sec-WebSocket-Key;
- use Sec-WebSocket-Key as a per-handshake connection identifier where available;
- fall back to a generated GUID if no connection identifier is available;
- maintain temporary connections before client identification;
- maintain identified clients after a valid SenderIdentifier is received;
- replace an existing identified client when a new connection announces the same SenderIdentifier;
- bind a connection to its Roof identifier when the first valid packet is received;
- enforce an unidentified connection timeout for clients that complete the WebSocket handshake but do not send a valid identifying packet within the heartbeat interval;
- accept only binary WebSocket messages containing serialised RoofWebSocketPacket JSON;
- drop malformed packets;
- drop circular routes where sender and forwarding target are the same;
- handle PathFinder heartbeat packets addressed to the PathFinder identifier;
- forward packets to destination clients where the target identifier is currently connected;
- rewrite SenderFingerprint before forwarding packets;
- send relay heartbeats to each identified client every 5 seconds;

- evaluate identified-client heartbeat state against a 15-second timeout, subject to the relay-side fallback limitation described in 7.4;
- attempt graceful closure of identified and temporary WebSocket connections during shutdown, using a five-second timeout per socket and aborting sockets when necessary;
- clean up socket references when clients close, timeout, or abort.
- expose PathFinder version information for runtime and release tracking;

2.1 Out-of-Scope Responsibilities

PathFinder should not be treated as:

- a permanent data store;
- an offline message queue;
- a central data owner;
- a payload inspection service;
- an application-level authentication authority unless explicitly implemented later;
- a source of truth for user-owned data;
- a replacement for local-first communication where direct local communication is available.

3. System Architecture

3.1 High-Level Architecture

PathFinder is a remote relay service positioned between Roof clients that cannot directly reach each other.

At a high level:

Roof Client A
|
WebSocket-over-TCP
v
PathFinder
|
WebSocket-over-TCP
v
Roof Client B

PathFinder receives packets from one connected client, determines the intended destination using the packet's forwarding identifier, and forwards the packet to the currently connected target client.

PathFinder does not parse the application-level payload beyond recognising heartbeat packets.

3.2 Process Startup

Program.Main controls the process lifecycle.

Startup behaviour:

1. Log the informational version and compile configuration.
2. Register Ctrl+C and process-exit handlers.
3. Start WebSocketRelay.Start() on a background thread.
4. Keep the main process alive using a one-second delay loop until exit is requested

On Ctrl+C:

1. Log the shutdown request.
2. Set the exit-requested flag.
3. Prevent immediate console termination.
4. Call WebSocketRelay.StopAsync().GetAwaiter().GetResult().

On process exit:

1. Log final cleanup.
2. Call WebSocketRelay.Stop(), which invokes StopAsync() synchronously.
3. Clear the relay thread reference.

The relay uses an atomic shutdown guard so repeated shutdown calls do not start duplicate shutdown sequences.

Shutdown stops the listener before attempting connection closure. Identified and temporary WebSocket connections receive bounded close attempts before the registries are cleared. See 11.7 for the detailed shutdown sequence.

3.3 Relay Runtime Model

WebSocketRelay.Start() controls the active relay runtime.

Runtime behaviour:

1. Start a static TcpListener on port 8071.
2. Create a cancellation token source for background relay tasks.
3. Start the relay heartbeat task.
4. Start the disconnection detection task.
5. Accept TCP clients in a loop.
6. Dispatch each accepted client to HandleClient.

3.4 Client Handling Flow

HandleClient() performs the WebSocket setup and connection registration.

Client handling flow:

1. Read the initial HTTP request from the network stream.
2. Check for WebSocket upgrade headers.
3. Parse request headers into a case-insensitive dictionary.
4. Build the WebSocket handshake response.
5. Write the handshake response to the network stream.
6. Wrap the existing stream with WebSocket.CreateFromStream.
7. Store a temporary PathFinderClient in _connectionsById.

8. Start ForwardAllPackets for the socket.

3.5 Client Registry Model

PathFinder maintains two client registries.

3.5.1 Temporary Connection Registry

_connectionsById: ConcurrentDictionary<string, PathFinderClient>

Purpose:

- store accepted WebSocket connections before they announce an application-level Roof identifier.

Key:

- connection ID derived from Sec-WebSocket-Key, or;
- generated GUID if required.

Temporary connections track creation time so PathFinder can enforce an unidentified connection timeout.

A temporary connection must send a valid identifying packet within the heartbeat interval. If it does not, PathFinder closes the socket, removes the connection from _connectionsById, and logs the timeout event.

3.5.2 Identified Client Registry

_clientsByIdentifier: ConcurrentDictionary<string, PathFinderClient>

Purpose:

- act as the authoritative routing table for relay forwarding.

Key:

- RoofWebSocketPacket.SenderIdentifier.

A connection becomes an identified client after the first valid packet containing SenderIdentifier is received.

When a new connection announces a SenderIdentifier that already exists in _clientsByIdentifier, the new connection replaces the old connection.

The replacement connection becomes the authoritative active client for that SenderIdentifier. The old socket is closed where safely possible, stale references are removed, and the reconnect/session replacement is logged.

3.6 Main Modules

Module	Responsibility
Roof.PathFinder/Program.cs	Process lifecycle, shutdown hooks, relay thread startup.
Roof.PathFinder/WebSocketRelay.cs	WebSocket handshake, client registry, heartbeat, timeout detection, packet forwarding.
Roof.Common/Networking/RNMS/WebSocket/RoofWebSocketPacket.cs	Relay packet DTO and JSON source-generation context.
Roof.Common/Networking/RNMS/WebSocket/WebSocketProvider.cs	Client-side WebSocket connection, heartbeat send/receive, packet deserialisation.
Roof.Common/Networking/RNMS/NetworkMessageProvider.cs	Facade for selecting TCP, UDP, or WebSocket providers.
Roof.Common/Tools/Logger.cs	Console and file logging utility used by PathFinder.

4. Data Model

4.1 PathFinderClient

PathFinderClient is the internal relay-side client state object.

Field	Type	Meaning
Identifier	string?	Application-level Roof identifier. Set after the first valid packet.
ConnectionId	string	Per-handshake connection identifier, usually derived from Sec-WebSocket-Key.
Socket	WebSocket?	Active WebSocket wrapper around the accepted TCP stream.
OriginIP	string?	Optional Cloudflare-originated client IP header.
RayId	string?	Optional Cloudflare ray/debug header.
LastHeartbeatSentTime	DateTime?	Last relay heartbeat sent to this client.

Field	Type	Meaning
LastHeartbeatReceivedTime	DateTime?	Last heartbeat received from this client.
CreatedAtUtc	DateTime	UTC creation timestamp, initialised to DateTime.UtcNow. Used to evaluate the identification deadline for temporary connections.

4.2 RoofWebSocketPacket

RoofWebSocketPacket is the shared relay packet class defined in Roof.Common.

Field	Type	Meaning
SenderIdentifier	string?	Logical sender identifier. Used by PathFinder to bind and route client state.
ForwardingToIdentifier	string?	Logical destination identifier used for relay routing.
SenderFingerprint	string?	PathFinder-assigned source connection fingerprint before forwarding.
Payload	string?	Caller-defined payload. PathFinder only special-cases WS_HEARTBEAT.

Serialisation behaviour:

- source-generation context: RoofWebSocketPacketJSONContext;
- JSON options: WriteIndented = false;
- WebSocket transport format: binary frame containing UTF-8 JSON bytes.

4.3 Native Request Packet

RoofNetworkNativeRequestPacket contains:

- ClientID;
- RoofIP;
- Requests.

Serialisation context:

RoofNetworkNativeRequestPacketJSONContext

4.4 Native Response Packet

RoofNetworkNativeResponsePacket contains:

- Status;
- Responses.

Behaviour:

- AddResponse() lazily initialises Responses;
- duplicate response strings are suppressed.

Serialisation context:

RoofNetworkNativeResponsePacketJSONContext

5. Communication Model

5.1 General Communication Flow

PathFinder communication follows this general model:

1. Client establishes TCP connection.
2. Client sends HTTP WebSocket upgrade request.
3. PathFinder validates and completes the WebSocket handshake.
4. Client sends a first valid RoofWebSocketPacket.
5. PathFinder binds the connection to SenderIdentifier.
6. Client sends destination-bound packets.
7. PathFinder validates the packet.
8. PathFinder looks up the destination client.
9. PathFinder forwards the packet if the destination is connected.
10. PathFinder drops the packet if the destination is unavailable.

5.2 Connection Binding

PathFinder does not know the client's application-level identifier during the handshake.

The first identifying packet must deserialise as RoofWebSocketPacket and contain nonempty SenderIdentifier, ForwardingToIdentifier and Payload values. PathFinder binds the connection to the announced SenderIdentifier before evaluating circular routing or processing the packet as a heartbeat or ordinary relay message.

Once received, BindConnectionIfFirstPacket() moves the connection from the temporary connection registry into the identified client registry.

A heartbeat packet may act as the first valid identifying packet, provided it contains a valid SenderIdentifier and is addressed to the PathFinder identifier.

If no valid identifying packet is received within the heartbeat interval, the connection is closed and removed from the temporary connection registry.

5.3 Duplicate Identifier Behaviour

If a new connection announces a SenderIdentifier that already exists in _clientsByIdentifier, PathFinder replaces the old connection with the new connection.

This behaviour is implemented because a duplicate identifier is most likely caused by the previous connection being dropped, stale, interrupted, or failing to clean up correctly.

Implemented behaviour:

- the new connection becomes the authoritative active connection for that SenderIdentifier;
- the old client entry is removed from _clientsByIdentifier;
- the old WebSocket is closed safely where possible;
- the old connection is removed from _connectionsById if still present;
- stale socket references are cleaned up;
- the replacement event is logged as a reconnect/session replacement event;
- duplicate replacement is not treated as a security violation under the current trust model.

This behaviour may be revisited once a stronger identifier ownership or session authentication model is introduced.

5.4 Routing Flow

For non-heartbeat packets:

1. PathFinder checks ForwardingToIdentifier.
2. PathFinder looks up the destination in _clientsByIdentifier.
3. Destination socket must exist.
4. Destination socket must be in WebSocketState.Open.
5. PathFinder logs the routing event.
6. PathFinder overwrites SenderFingerprint with the source connectionId.
7. PathFinder serialises the updated packet.
8. PathFinder sends the packet as a binary WebSocket message to the destination socket.

If the target is not connected, PathFinder logs a warning and drops the packet.

5.5 Payload Handling

PathFinder does not interpret ordinary application payloads.

PathFinder only special-cases payloads where: Payload = WS_HEARTBEAT and the packet is addressed to the PathFinder identifier.

All other payloads are treated as relay payloads and forwarded according to the routing model.

6. PathFinder Networking Interface

6.1 Listener Configuration

Setting	Value
Listen address	IPAddress.Any

Setting	Value
Listen port	8071
Listener type	TcpListener
WebSocket implementation	Manual HTTP upgrade + WebSocket.CreateFromStream
Accepted message type	Binary WebSocket messages
Packet encoding	UTF-8 JSON bytes

6.2 WebSocket Handshake Requirements

PathFinder requires the initial request to contain:
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key

Handshake response:
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: {base64(sha1(key + websocket-guid))}

Standard WebSocket GUID: 258EAFA5-E914-47DA-95CA-C5AB0DC85B11

6.3 Cloudflare Header Handling

PathFinder extracts the following optional headers:
CF-Connecting-IP
CF-Ray
Sec-WebSocket-Key

These are used for logging and connection metadata.

Current behaviour does not validate that the request actually came through Cloudflare or a trusted reverse proxy.

6.4 Packet Validation

PathFinder drops a received packet when:

- deserialisation fails;
- packet is null;
- SenderIdentifier is null or empty;
- ForwardingToIdentifieris null or empty;
- Payload is null or empty;
- SenderIdentifier and ForwardingToIdentifier are equal.

PathFinder logs malformed and circular packets.

6.5 Unsupported Message Types

PathFinder currently accepts binary WebSocket messages containing UTF-8 JSON.

Non-binary WebSocket messages are ignored.

A future decision is required on whether unsupported message types should be silently ignored or explicitly rejected with a WebSocket close status.

Heartbeat packets are treated as valid identifying packets. Because the heartbeat itself contains the client identifier and destination PathFinder identifier, a newly connected client may identify itself through its first heartbeat packet.

A connection that does not send a valid identifying packet within the heartbeat window is closed and removed from the temporary connection registry.

7. Heartbeat & Disconnection Model

7.1 Constants

Concern	Value
PathFinder identifier	43627a1de2ca425e9bf877b80dc45725
WebSocket heartbeat payload	WS_HEARTBEAT
Heartbeat recurrence	5 seconds
Disconnection timeout	15 seconds
PathFinder heartbeat loop delay	250 ms
PathFinder disconnection loop delay	1000 ms

7.2 Relay-to-Client Heartbeat

PathFinder sends a heartbeat to each identified client every 5 seconds.

Heartbeat packet shape:

```
1. {
2.   "SenderIdentifier": "43627a1de2ca425e9bf877b80dc45725",
3.   "ForwardingToIdentifier": "{client.Identifier}",
4.   "Payload": "WS_HEARTBEAT"
5. }
```

The packet is serialised through RoofWebSocketPacketJSONContext and sent as a binary WebSocket frame.

7.3 Client-to-Relay Heartbeat

A heartbeat sent to PathFinder is detected when: ForwardingToIdentifier == PathFinder identifier and: Payload == WS_HEARTBEAT

When detected, PathFinder updates the sender client's LastHeartbeatReceivedTime and does not forward the packet.

7.4 Disconnection Detection

PathFinder evaluates identified clients approximately once per second.

An identified client is removed from the routing registry when its socket is null, Closed or Aborted.

For heartbeat timeout evaluation, the relay selects a timestamp in this order:

1. LastHeartbeatReceivedTime.
2. LastHeartbeatSentTime.
3. DateTime.UtcNow.

If the selected timestamp is more than 15 seconds old, PathFinder attempts to close the socket with EndpointUnavailable and removes the client from the identified registry.

LastHeartbeatReceivedTime is updated only when a packet is addressed to the PathFinder identifier and its payload is WS_HEARTBEAT. Ordinary forwarded application packets do not refresh this timestamp.

Relay Timeout Limitation

Before the first client heartbeat is received, the relay falls back to LastHeartbeatSentTime. That timestamp is refreshed whenever the relay successfully sends a heartbeat.

Consequently, an identified client that never sends its first heartbeat may remain connected while relay heartbeat sends continue to succeed. The current implementation does not guarantee a 15-second receive timeout for that case.

After a first client heartbeat has been received, the timeout is evaluated against that received timestamp.

This limitation belongs to the PathFinder relay. The client-side Roof.Common WebSocketProvider separately initialises its received-heartbeat timestamp at connection establishment and can time out when its first relay heartbeat never arrives.

Temporary Connections

Temporary unidentified connections are evaluated using CreatedAtUtc.

A connection becomes eligible for identification-timeout closure when more than five seconds have elapsed without a valid identifying packet. The monitor attempts to close it with PolicyViolation and the reason "Identifying packet timeout.", then removes its temporary registry entry.

The five-second threshold is evaluated by the periodic monitor. It is not an exact wall-clock guarantee that socket closure completes within five seconds.

8. Security & Trust Boundaries

8.1 Relay Security Position

PathFinder is currently a relay and coordination component.
It does not currently implement explicit relay-boundary authentication.

Current PathFinder behaviour trusts:

- the SenderIdentifier supplied by the first valid packet;
- the ForwardingToIdentifier supplied by the sender;
- the WebSocket connection as the active holder of that identifier.

No current implementation was identified for:

- token validation;
- signed packet validation;
- per-client secret verification;
- origin validation;
- path validation;
- authorisation checks that prove a sender may route to a destination.

Client ownership proof for SenderIdentifier is not yet formally defined.

Current decision: SenderIdentifier ownership proof remains as-is for now and is to be explored later.

For the current implementation phase, PathFinder continues to trust the first valid packet or heartbeat that announces a SenderIdentifier.

This is an accepted temporary limitation, not the final security model.

Future exploration should define whether identifier ownership is proven through:

- signed packets;
- client secrets;
- certificates;
- token-based relay sessions;
- Roof-native identity material;
- another Roof-specific trust mechanism.

8.2 Payload Confidentiality

PathFinder does not encrypt or decrypt packet payloads.
PathFinder forwards the serialised packet and only special-cases relay heartbeats.
Therefore, end-to-end confidentiality must be implemented by callers at the payload layer.

Roof Common provides AES-256 helper utilities for:

- AES key generation;
- IV generation;
- raw AES encryption/decryption with PKCS7 padding;
- network encryption/decryption helpers that convert plaintext to UTF-8, encrypt with AES, Base64 encode, and send Base64 text bytes.

8.3 SenderFingerprint

Before forwarding, PathFinder overwrites: SenderFingerprint = connectionId

This provides the recipient with a relay-assigned connection-level fingerprint.

However, the current fingerprint is derived from Sec-WebSocket-Key where available, which is handshake-specific and should not be treated as long-term identity proof.

The SenderFingerprint should therefore be treated as diagnostic or relay-session metadata unless a stronger future identity model is implemented.

8.4 TLS and Proxy Boundary

PathFinder is intended to run behind Cloudflare in the current deployment model.

Current decision: PathFinder is always behind Cloudflare for the current deployment model.

Direct public access to the PathFinder process should therefore be blocked or restricted by infrastructure controls wherever possible.

The application process itself currently does not terminate TLS. If production clients connect using: wss://roofpathfinder.dflb.org/ TLS is expected to be terminated upstream by Cloudflare or equivalent infrastructure.

PathFinder currently reads Cloudflare headers such as:

- CF-Connecting-IP
- CF-Ray

These headers are useful for logging and diagnostics, but production deployment should ensure they are only trusted when traffic is confirmed to have passed through Cloudflare or another approved proxy layer.

In the far future, PathFinder may evolve beyond the Cloudflare-backed model into a distributed Roof-native edge relay model. This future model would replicate Cloudflare Tunnel-like behaviour within the Roof network itself, allowing Roof-operated distributed edge nodes to provide relay and tunnel functionality without depending permanently on Cloudflare.

Future deployment models may therefore include:

Model	Description
Cloudflare-backed relay	Current intended production model. PathFinder sits behind Cloudflare.
Roof-native edge relay	Future distributed edge model inside the Roof network.
Hybrid transition model	Cloudflare remains the public entry point while Roof-native relay nodes are gradually introduced.

8.5 Security Direction Consideration

Future security hardening should define:

- client authentication model;
- identifier ownership proof;
- relay authorisation model;
- session migration rules;
- payload encryption expectations;
- proxy trust model;
- TLS termination responsibilities;
- replay protection requirements;
- maximum packet size;
- abuse/rate-limiting controls.

9. Networking Protocols

PathFinder's primary relay interface is WebSocket-over-TCP.

The current implementation:

- receives raw TCP connections;
- manually upgrades to WebSocket;
- uses WebSocket.CreateFromStream;
- accepts binary WebSocket frames;
- expects UTF-8 JSON serialised RoofWebSocketPacket objects.

10. Scalability Model

10.1 Current Scalability Model

PathFinder currently uses in-memory concurrent dictionaries for connection and client state.

Current state is local to the process:
_connectionsById
_clientsByIdentifier

This means a single PathFinder process is responsible for knowing which clients are connected to that process.

10.2 Horizontal Scaling Considerations

Horizontal scaling is not fully defined in the current implementation.
Because client state is process-local, horizontal scaling would require one or more of the following:

- sticky sessions at the load balancer or proxy layer;
- shared distributed registry of connected client identifiers;
- inter-PathFinder routing between relay nodes;
- external connection/session coordination layer;
- deterministic routing of client identifiers to relay nodes.

Without one of these, two clients connected to different PathFinder instances may not be routable to each other.

10.3 Load Distribution

Current load distribution is expected to be handled outside the process if multiple PathFinder instances are deployed.

Possible future models:

Model	Description
Single relay node	Simplest model. All remote relay traffic goes through one PathFinder instance.
Sticky load-balanced nodes	Clients remain attached to the same relay instance.
Distributed relay mesh	PathFinder instances communicate with each other to route packets.
Regional relay nodes	Clients connect to geographically closer PathFinder instances.

10.4 Connection Limits

No explicit connection limit is currently defined.

Future specification should define:

- maximum concurrent connections;
- maximum temporary connections;
- maximum identified clients;
- maximum per-client packet rate;
- maximum packet size;
- maximum heartbeat failure tolerance;
- expected memory consumption per connection.

11. Failure Handling

11.1 Startup Failure

If the relay listener starts successfully, PathFinder logs readiness and begins accepting clients.

If AcceptTcpClientAsync() throws:

- OperationCanceledException; or
- SocketException;

the accept loop exits quietly.
Other startup or accept exceptions are not currently surfaced through an explicit failure management model.

11.2 Handshake Failure

For client handshake failure:

- non-WebSocket requests are closed immediately;
- missing Sec-WebSocket-Key causes handshake failure and connection close;
- malformed headers are tolerated except where required checks fail.

11.3 Packet Receive Failure

PathFinder behaviour on receive failure:

- deserialisation errors are logged;
- the connection remains open after deserialisation failure;
- malformed packets are logged and dropped;
- non-binary WebSocket messages are ignored;
- close frames trigger socket cleanup.

11.4 Routing Failure

PathFinder is intentionally fire-and-forget.

Current decision: PathFinder does not provide ACK, retry, or offline queueing for forwarded WebSocket relay packets.

This is intentional because PathFinder should not block incoming connections or already-connected clients while waiting for delivery confirmation, retry handling, or offline destination availability.

PathFinder behaviour on routing failure:

- missing destination clients cause warning logs;
- packets are dropped when the destination is unavailable;
- no retry is attempted;
- no offline queue exists;
- no delivery acknowledgement exists for forwarded WebSocket packets;

- no blocking wait occurs for destination availability;
- no backpressure mechanism currently exists for slow recipients.

Delivery reliability, if required, should be implemented at a higher application or protocol layer rather than inside the PathFinder relay itself.

11.5 Client Close Path

Client close flow:

1. Relay receive loop reads a WebSocket close frame.
2. Relay logs the disconnection.
3. Relay calls SafeCloseAndCleanup.
4. Relay closes the socket if still open.
5. Relay removes the temporary connection by connection ID.
6. Relay removes any identified client entries pointing to that socket or connection ID.

11.6 Heartbeat Timeout Path

The relay periodically sends WS_HEARTBEAT packets to identified clients. Clients independently send WS_HEARTBEAT packets addressed to the PathFinder identifier.

The disconnection monitor evaluates each identified client using the timestamp fallback described in 7.4.

When the selected timestamp is more than 15 seconds old, the relay attempts to close the socket with EndpointUnavailable and removes its identified registry entry.

Ordinary application traffic does not refresh LastHeartbeatReceivedTime.

The relay's sent-heartbeat fallback can prevent timeout before the first client heartbeat has been received. This remains an implementation limitation and must not be described as guaranteed receive-timeout enforcement.

11.7 Shutdown Behaviour

WebSocketRelay.StopAsync() performs the relay shutdown sequence. Stop() is a synchronous wrapper around that method.

An atomic guard prevents repeated shutdown calls from starting additional shutdown sequences.

Shutdown proceeds as follows:

1. Request cancellation of the relay heartbeat and disconnection-monitor tasks.
2. Stop the TCP listener to prevent further connection acceptance.
3. Snapshot the identified-client registry and the temporary unidentified-connection registry.
4. Start close attempts for the captured connections, avoiding duplicate attempts for the same connection ID.

5. For sockets in Open or CloseReceived state, attempt a WebSocket close handshake using EndpointUnavailable and the reason “PathFinder shutting down.”
6. Apply a five-second timeout to each socket’s close attempt.
7. Abort sockets that fail to reach Closed state or whose close attempt times out or fails.
8. Wait for the close-attempt tasks.
9. Clear both connection registries.
10. Log the close-result summary.
11. Dispose the listener.

The summary records attempted, closed, already-closed, aborted and failed outcomes.

The five-second value is a per-socket close timeout. It must not be represented as a guaranteed total process-shutdown duration.

12. Performance Considerations

12.1 Latency

PathFinder is a relay and therefore adds one remote hop between communicating clients.

Latency depends on:

- client-to-PathFinder network latency;
- PathFinder-to-destination network latency;
- relay processing overhead;
- WebSocket frame handling;
- JSON serialisation/deserialisation;
- destination socket availability.

No explicit latency target is currently defined.
Future target categories:

Category	Suggested Definition
Local direct communication	Not PathFinder-routed; should remain preferred.
Remote relay communication	Acceptable for control, coordination, and moderate data messages.
High-volume transfer	Should be assessed carefully before routing through PathFinder.

12.2 Throughput

PathFinder forwards binary WebSocket packets containing JSON payloads.

Throughput considerations include:

- WebSocket frame size;

- JSON serialisation cost;
- memory allocation during receive buffer accumulation;
- destination socket write performance;
- lack of backpressure handling;
- lack of packet size limit.

No explicit throughput target is currently defined.

12.3 Heartbeat Overhead

Heartbeats are sent every 5 seconds to each identified client. Heartbeat overhead increases linearly with connected clients.

Current heartbeat behaviour should be reviewed under load, especially for:

- large numbers of connected clients;
- slow or unstable connections;
- repeated disconnection/reconnection cycles.

12.4 Heartbeat Loop Scheduling

The client-side WebSocketProvider.SendHeartbeat() includes a one-second delay on every loop iteration and uses provider-owned cancellation. The previously documented heartbeat busy-loop defect is no longer present.

The PathFinder relay evaluates heartbeat sending approximately every 250 milliseconds and disconnection state approximately every one second.

Heartbeat scheduling, cancellation and behaviour with slow or unstable connections remain validation areas. They should not be listed as an existing client-side no-delay busy-loop defect.

12.5 Packet Size

The maximum allowed WebSocket relay packet size is currently undefined.

Current decision: Maximum WebSocket relay packet size is to be explored.

PathFinder should continue operating without a formal packet-size limit for the current phase, but this must be revisited before production hardening or wider beta use.

Future specification should define:

- maximum binary WebSocket message size;
- maximum serialised RoofWebSocketPacket size;
- behaviour for oversized packets;
- whether oversized packets are dropped or cause socket closure;
- whether packet limits differ between IA, Beta, and Release configurations;
- logging behaviour for oversized packet attempts.

Until this is defined, packet-size handling remains a known limitation.

13. Deployment & Runtime Model

13.1 Pathfinder Project Configuration

Setting	Value
SDK	Microsoft.NET.Sdk
Output type	Exe
Target framework	net10.0
Nullable	Enabled
Implicit usings	Enabled
NativeAOT	PublishAot=true
Globalization	Invariant globalisation enabled
Configurations	Debug, Release, IA, Beta
References	Roof.Common

13.2 Network Exposure

PathFinder binds to: IPAddress.Any:8071 This exposes the listener on all available interfaces at the process level.

However, the current intended deployment model requires PathFinder to sit behind Cloudflare. Public access should be mediated through Cloudflare, and direct access to the raw PathFinder listener should be blocked or restricted by infrastructure controls wherever possible.

Production deployment should define:

- Cloudflare Tunnel or reverse proxy configuration;
- firewall rules preventing direct public access to port 8071;
- TLS termination at Cloudflare or approved upstream infrastructure;
- trusted proxy header handling;
- rate limiting;
- DDoS protection expectations;
- logging retention;
- process supervision;
- restart policy;
- deployment target environment.

Future direction:

PathFinder may later evolve into a distributed Roof-native edge relay model, where Roof-operated nodes provide relay/tunnel behavior similar to Cloudflare Tunnel, but within the Roof network itself.

13.3 Update Strategy

PathFinder supports versioning. The update strategy should use the PathFinder version number to track runtime behaviour, deployment state, and compatibility expectations.

Future specification should define:

- rollout process for IA, Beta, and Release configurations;
- rollback procedure;
- checksum/signature verification;
- outage communication process;
- migration path for wire-format changes;
- version compatibility expectations between PathFinder and Roof.Common;
- version reporting behaviour for runtime diagnostics.

13.4 Versioning

PathFinder supports formal versioning.

The current assembly metadata distinguishes assembly/file versions from informational versions:

Version field or configuration	**Current value**
AssemblyVersion	0.0.0.0
AssemblyFileVersion	0.0.0.0
Debug informational version	0.0.0.0-debug
IA informational version	1.0.0.0-ia
Beta informational version	1.0.0.0-beta
Release informational version	0.0.0.0-release

The runtime Version.Get() helper reads the informational version from the process entry assembly. Program.Main logs that value together with the compile configuration at startup.

These values describe the current source metadata. They do not establish that every configuration has been published as a release.

Versioning is used to identify the current PathFinder build, support release tracking, and distinguish behaviour across internal, beta, and release builds.

The versioning model should follow semantic versioning:
Major.Minor.Patch.Build

- Major version increments are used for breaking protocol, relay, deployment, or compatibility changes.

- Minor version increments are used for new backward-compatible features or behaviour additions.
- Patch version increments are used for bug fixes, hardening, and internal corrections.
- Build version increments are used for minor patches that shouldn't affect patch version. For example, fixing a spelling error.
- IA, Beta, and Release builds may use the same semantic version number with different build configuration labels. For example, 1.0.0.0-ia.
- Version information should be visible in startup logs.
- Version information should be included in deployment records and release notes.
- Future protocol-level version negotiation may be added if required.

14. Observability

14.1 Logging Utility

PathFinder uses Roof.Common.Tools.Logger.

Console logging behaviour:

- neutral messages use cyan;
- warnings use yellow;
- danger messages use red;
- when a debugger is attached, output goes to Debug.WriteLine;
- otherwise output goes to Console.WriteLine.

File logging behaviour:

- directory: {current working directory}/Logs;
- file name: process start time formatted as yyyy-MM-ddTHHmmss.log;
- LogToFile() writes in all builds;
- LogToFile() mirrors to console in DEBUG;
- LogExceptionToFile() writes exception metadata and stack trace.

14.2 PathFinder Log Events

PathFinder logs:

- startup on using the WEBSOCKET_TCP_LISTENER_PORT constant;
- handshake completion;
- connection ID;
- origin IP;
- Cloudflare ray ID;
- handshake failure;
- packet deserialisation failure;
- malformed packet drops;
- circular route drops;
- routing events;
- missing target warnings;
- heartbeat timeout disconnections;
- socket close/disconnect events.

- duplicate SenderIdentifier replacement events;
- unidentified connection timeout events;
- graceful shutdown close attempts;
- forced socket aborts during shutdown;
- shutdown cleanup summary;
- PathFinder version at startup.

14.3 Metrics Consideration

Future observability should include metrics for:

- active temporary connections;
- active identified clients;
- total accepted connections;
- failed handshakes;
- malformed packet count;
- circular route drop count;
- forwarded packet count;
- missing-target drop count;
- heartbeat timeout count;
- average connection duration;
- average packet forwarding latency;
- bytes received;
- bytes sent;
- process memory usage;
- process CPU usage.

14.4 Payload Logging Boundary

PathFinder should not log application payload contents unless explicitly permitted for a controlled debugging mode.

Rule consideration: PathFinder may log routing metadata, connection state, and errors, but must not log user/application payload data by default.

This preserves PathFinder's relay-only role and reduces privacy risk.

15. Testing & Validation

No tests are present. Testing should be added before production hardening.

15.1 Validation Areas

- WebSocket handshake success;
- WebSocket handshake failure;
- missing Sec-WebSocket-Key;
- first-packet identifier binding;
- duplicate identifier behaviour;
- relay forwarding to connected target;

- missing-target drop behaviour;
- circular-route drop behaviour;
- heartbeat send behaviour;
- heartbeat receive behaviour;
- Relay timeout cleanup after a received client heartbeat becomes more than 15 seconds old.
- Relay behaviour when an identified client never sends its first heartbeat, including the current sent-heartbeat fallback limitation.
- temporary unbound connection cleanup;
- close-frame cleanup;
- malformed packet handling;
- binary-only packet handling;
- unsupported text-frame behaviour;
- packet serialisation compatibility;
- duplicate identifier replacement behaviour;
- old socket cleanup after duplicate identifier replacement;
- new socket authority after duplicate identifier replacement;
- unidentified connection timeout cleanup;
- heartbeat-as-first-identifying-packet behaviour;
- graceful shutdown of identified clients;
- graceful shutdown of temporary unidentified clients;
- forced shutdown timeout behaviour;
- version logging at startup.
- Client-side WebSocketProvider times out when no initial relay heartbeat arrives.
- Shutdown stops the listener before connection-close attempts.
- Shutdown applies a five-second timeout per socket and aborts sockets when closure does not complete.

15.2 Network Simulation

Network testing should include:

- client disconnect;
- PathFinder restart;
- destination offline;
- source offline;
- slow destination;
- high packet volume;
- unstable network;
- delayed heartbeat response;
- duplicate client identifier;
- invalid packet spam;
- proxy header absence;
- proxy header spoofing.

15.3 Load Testing

Load testing should define and measure:

- maximum concurrent connected clients;
- maximum temporary connections;
- average CPU usage;
- average memory usage;
- packet forwarding throughput;
- packet forwarding latency;
- behaviour under heartbeat storms;
- behaviour under malformed packet floods;
- behaviour under many missing-target packets.

15.4 Acceptance Criteria

Minimum acceptance criteria should include:

- PathFinder starts and listens on port 8071;
- valid WebSocket clients can connect;
- valid clients can register identifiers;
- connected clients can exchange packets through PathFinder;
- malformed packets are dropped without crashing the process;
- disconnected clients are cleaned up;
- Relay heartbeat timeout removes clients whose received-heartbeat timestamp becomes stale. The current no-initial-client-heartbeat fallback limitation is explicitly documented and tested.
- missing destinations do not crash the relay;
- payloads are not inspected or logged by default;
- relay can be stopped cleanly;
- NativeAOT build succeeds.
- duplicate SenderIdentifier connections replace the old active connection;
- stale socket references are removed after duplicate replacement;
- unidentified temporary connections are closed after the heartbeat interval;
- heartbeat packets can identify a newly connected client;
- PathFinder shutdown attempts to close active identified sockets gracefully;
- PathFinder shutdown attempts to close temporary unidentified sockets gracefully;
- PathFinder can force-abort sockets that do not close within the shutdown timeout;
- PathFinder logs its version at startup.

16. Risks & Limitations

16.1 Security Risks

Current risks:

- PathFinder has no explicit relay-level authentication;
- SenderIdentifier ownership proof is not yet formally defined;
- SenderIdentifier is trusted from client-supplied packet or heartbeat data;
- no authorisation check confirms whether a sender may route to a destination;
- duplicate identifier replacement is intentionally allowed for reconnect handling but may require stronger controls later;
- SenderFingerprint is not a long-term identity proof;

- application-level TLS is absent from the relay process;
- PathFinder is intended to sit behind Cloudflare, but application-level proxy trust validation is not yet implemented;
- AES helper methods require callers to manage key and IV storage/rotation correctly.

16.2 Reliability Risks

Current residual risks:

- duplicate identifier replacement is implemented, but should be validated under unstable network and rapid reconnect scenarios;
- unidentified connection timeout is implemented, but timeout value may need tuning under slow or high-latency environments;
- graceful shutdown is implemented, but socket close behaviour should be tested under hung, aborted, or slow clients;
- no offline queue exists by design;
- no delivery acknowledgement exists for forwarded WebSocket packets by design;
- no retry exists for failed relay delivery by design;
- no blocking wait occurs for disconnected destinations by design;
- no backpressure mechanism exists for slow recipients.

The lack of ACK, retry, and offline queueing is intentional for the current PathFinder model. PathFinder should remain lightweight and should not block incoming or active connected clients waiting for delivery confirmation.

16.3 Performance Risks

Current risks:

- no explicit PathFinder packet size limit;
- receive buffer accumulation may grow depending on message behaviour;
- high heartbeat volume may become expensive at scale;
- JSON serialisation overhead may become significant for high-throughput use.

16.4 Protocol Compatibility Risks

Current risks:

- Peers using different destination-field spellings may fail to route or validate packets correctly.
- No protocol-version negotiation is implemented.
- Heartbeat markers and the PathFinder identifier must remain consistent between the relay and connected components.
- Packet-contract changes require coordinated compatibility assessment across PathFinder, Roof.Common, Engine and client applications.

16.5 Implementation Completeness Risks

Current risks:

- no tests are found for PathFinder;
- implemented duplicate identifier replacement requires integration and reconnect testing;

- implemented unidentified connection timeout requires network-condition testing;
- implemented graceful shutdown requires socket lifecycle testing;
- WebSocketProvider.Initialize() returns false when connection establishment fails, but does not expose structured failure details. Callers must inspect the Boolean result.
- heartbeat timeout behaviour depends on heartbeat state initialisation;
- production deployment assumptions are not fully specified;
- versioning exists but release and compatibility policy still need further definition.

Appendices

Appendix A: Important Constants

Constant	Value	Location
PathFinder identifier	43627a1de2ca425e9bf877b80dc45725	Program, WebSocketProvider
PathFinder listen port	8071	WebSocketRelay
WebSocket heartbeat payload	WS_HEARTBEAT	WebSocketRelay, WebSocketProvider
Managed TCP data prefix	MTCP_DATA:	ManagedTCPProvider
Managed TCP heartbeat	MTCP_HEARTBEAT	ManagedTCPProvider
Managed UDP data prefix	MUDP_DATA:	ManagedUDPProvider
Managed UDP ACK prefix	MUDP_ACK:	ManagedUDPProvider
Managed UDP heartbeat	MUDP_HEARTBEAT	ManagedUDPProvider
Unmanaged UDP data prefix	UMUDP_DATA:	UnmanagedUDPProvider
Unmanaged UDP sent heartbeat	UMUDP_HEARTBEAT	UnmanagedUDPProvider
Unmanaged UDP received heartbeat check	UMUDP_HEARTBEAT	UnmanagedUDPProvider
Default heartbeat recurrence	5 seconds	TCP, UDP, WebSocket, PathFinder
Default disconnection timeout	15 seconds	TCP, UDP, WebSocket, PathFinder
Managed UDP retransmission interval	2 seconds	ManagedUDPProvider
Managed UDP max retries	5	ManagedUDPProvider
Managed TCP max frame size	64 MiB	ManagedTCPProvider
Unidentified connection timeout	5 seconds / heartbeat interval	WebSocketRelay
Shutdown close timeout per socket	5 seconds	WebSocketRelay

Constant	Value	Location
PathFinder version	1.0.0.0	AssemblyInfo.cs
Debug informational version	0.0.0.0-debug	AssemblyInfo.cs
IA informational version	1.0.0.0-ia	AssemblyInfo.cs
Beta informational version	1.0.0.0-beta	AssemblyInfo.cs
Release informational version	0.0.0.0-release	AssemblyInfo.cs

Appendix B: Glossary

Term	Meaning
PathFinder	Roof remote relay process for WebSocket-routed clients.
RNMS	Roof Network Messaging System, the provider abstraction in Roof.Common.
Identified client	A WebSocket connection that has announced a valid SenderIdentifier.
Temporary connection	A WebSocket connection that completed handshake but has not yet sent a valid identifying packet.
Sender identifier	Logical Roof client/device identifier supplied in RoofWebSocketPacket.
Forwarding identifier	Logical target identifier supplied in RoofWebSocketPacket.ForwardingToIdentifier.
Sender fingerprint	Relay-assigned connection ID added before forwarding.
Managed TCP	Length-prefixed TCP transport with heartbeats.
Managed UDP	UDP transport with sequence numbers, ACKs, retransmission, and heartbeats.
Unmanaged UDP	UDP transport without reliability guarantees.
Heartbeat	Periodic liveness packet used to detect active or stale connections.
Relay	A network component that forwards packets without owning or interpreting application data.

Appendix C: Relay Connection Flow

```
sequenceDiagram
    participant C as Client
    participant PF as PathFinder
    participant T as Target Client

    C->>PF: TCP connect
    C->>PF: HTTP Upgrade: websocket
    PF->>C: 101 Switching Protocols
    C->>PF: RoofWebSocketPacket(SenderIdentifier=C)
    PF->>PF: Bind connectionId to C
    T->>PF: Connect and bind target identifier
    C->>PF: Packet forwarding to T
    PF->>PF: Set SenderFingerprint = connectionId
    PF->>T: Forward binary RoofWebSocketPacket
```

Appendix D: Heartbeat Flow

```
sequenceDiagram
    participant PF as PathFinder
    participant C as Client

    loop Every 5 seconds
        PF->>C: WS_HEARTBEAT from PathFinder identifier
        C->>PF: WS_HEARTBEAT to PathFinder identifier
        PF->>PF: Update LastHeartbeatReceivedTime
    End
```

Appendix E: Component Diagram

```
flowchart LR
    ClientA["Remote Client A"] --> WSProviderA["Roof.Common WebSocketProvider"]
    ClientB["Remote Client B"] --> WSProviderB["Roof.Common WebSocketProvider"]
    WSProviderA --> PF["Roof.PathFinder WebSocketRelay :8071"]
    WSProviderB --> PF
    PF --> Registry["Identifier Registry"]
    PF --> Packet["RoofWebSocketPacket JSON"]
    Packet --> Common["Roof.Common"]
```